

CSI243: Functional programming

Defining functions

- **In Haskell every value has a type (remember syntactic type evaluation)**

- **Examples:-**

```
2.4 :: Float --- "2.4 has type Float"
```

```
`3' :: Char
```

```
(2, "Peter", "csi243") :: (Int, String, String) ---
```

- **Functions also have types**

- **Examples:-**

```
add :: Int->Int->Int ---type signature declaration
```

```
add :: Float -> Int -> Float ---
```

```
add :: Float -> Float -> Float ---
```

- **The basic types may not be sufficient for a given problem or problem domain**

CSI243

Defining functions

- **Defining type synonyms (or type aliases)**
 - **Could thus use own type declarations through definition of type aliases or type synonyms**

- **Examples:-**

```
type String = [Char] ---  
type Pos = (Int, Int) ---  
type Board = [Pos] ---
```

- **Type declaration may be parameterised by other types**

- **Example**

```
type Parser a =String ->[(a,String)]  
type Assoc k v =[(k,v)]
```

Defining functions **CSI243**

- **User-defined types**
 - **May define own types using the data declaration**
 - **Examples:-**

```
data Result = Pass | Fail ---data constructors
data TLight = Red | Green | Yellow ---
data Course = (title, code, credits) ---
```
- **Type declaration may be recursive**

```
data Tree a = Leaf a | Branch (Tree a) (Tree a)
```

Defining functions **CS1243**

- **Let and where expressions**
 - **The `where` clause makes definitions local to function definitions**
- **Example**
 - **Computation of area given perimeter and length of a rectangle**

$$\text{area} = l * w$$

$$\text{perimeter} = 2(l+w)$$

$$\text{where } w = \frac{\text{perimeter} - 2l}{2}$$

Defining functions **CSI243**

- **Let and where expressions**
 - **The `let` clause makes definitions local to an expression**
 - **Example**

- **Prior computation of area**

`let width = 0.3m in area = l * w`

- **Given `x` in `f(x)` function**

`let x = 3 in  $y = x^2$`

- **Course in a programme**

`let course = "csi304" in
isIn(Csprog, course)`

- **Student taking course**

`let course = "csi304" in
getStuds(offered(course))`

Defining functions **CS1243**

- **Recursion**
 - **A recursive function (or object) refers to the function (or object) itself in its definition**
- **Example**
 - **Specify the `fact(n)` which computes the factorial of a natural number `n`**
 - **Give the implementation `fact(n)`**
 - **H/w : Proof that `fact(n)` does what is intended**

Defining functions **CSI243**

- **Recursion**
 - **Example**
 - **Specify** `fact (n)`

n	fact(n)
0	$\begin{cases} n * \text{fact}(n - 1) & \text{iff } (n \geq 1) \\ 1 & \text{iff } n = 0 \end{cases}$
1	$\begin{cases} n * \text{fact}(n - 1) & \text{iff } (n \geq 1) \\ 1 & \text{iff } n = 0 \end{cases}$
i	$\begin{cases} n * \text{fact}(n - 1) & \text{iff } (n \geq 1) \\ 1 & \text{iff } n = 0 \end{cases}$
(i + 1)	$\begin{cases} n * \text{fact}(n - 1) & \text{iff } (n \geq 1) \\ 1 & \text{iff } n = 0 \end{cases}$

$$\text{fact } (n) = \begin{cases} n * \text{fact}(n - 1) & \text{iff } n > 0 & \text{--recursive step} \\ 1 & \text{iff } n = 0 & \text{--basis} \end{cases}$$

Defining functions **CS1243**

- **Recursion**

- **Give the** `fact (n)` **implementation**

```
fact :: Int -> Int
```

```
fact n
```

```
    | n == 0 = 1
```

```
    | n > 0 = n * fact (n-1)
```

- **H/w : Give the** `fact` **implementation without using guards**

- **Recursion**

- **Example**

- **But what if n is negative integer argument to the `fact`**

$$\text{fact}(n) = \begin{cases} 1 & \text{iff } n = 0 \\ \text{fact}(n-1) * n & \text{iff } n > 0 \\ \text{erroneous } n & \text{iff } n < 0 \end{cases}$$

- **Give the `fact` implementation**

```
fact :: Int -> Int
```

```
fact n
```

```
  | n == 0 = 1
```

```
  | n > 0 = fact (n-1) * n
```

```
  | n < 0 = error "fact not defined"
```

Defining functions

Pattern matching

- **Function definition whereby a sequence of syntactic expressions (called patterns) is used to choose between a sequence of results of the same type**
- **If the 1st pattern is matched, then the 1st result is chosen otherwise**
- **If the 2nd pattern is matched, the 2nd result is chosen,**

Defining functions

Pattern matching

CSI243

- **Example:-**

- **The** `negation` **operation may be defined as:-**

`¬ :: Bool -> Bool`

`¬False = True`

`¬True = False`

- **The** `∧` **operation may be defined as:-**

`(∧) :: Bool -> Bool -> Bool`

`True ∧ True = True`

`False ∧ True = False`

`True ∧ False = False`

`False ∧ False = False`

Defining functions **CS1243**

Pattern matching

- **Example:-**
 - **The definitions for the $\wedge$ operation may be simplified using the wildcard patterns to:-**

```
( $\wedge$ ) :: Bool -> Bool -> Bool
```

```
True  $\wedge$  b = b
```

```
False  $\wedge$  _ = False
```

- **The $\wedge$ operation may be simplified using the variables patterns to:-**

```
( $\wedge$ ) :: Bool -> Bool -> Bool
```

```
( $\wedge$ ) a b
```

```
  | a==b = b
```

```
  | otherwise = False
```

Defining functions

- **Pattern matching**
- **Tuple patterns**
 - **A tuple of patterns is a pattern, which matches any tuple of same arity whose components all match the corresponding patterns in order**
 - **Examples:-**
 - `Prelude's` `fst` **which gives the first component of a pair may be defined as:-**
$$\text{fst} :: (a, b) \rightarrow a$$
$$\text{fst } (x, _) = x$$

Defining functions

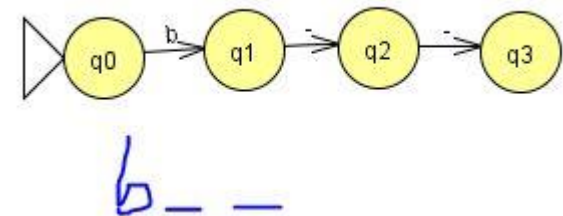
- **Pattern matching**
- **Tuple patterns**
 - **Examples:-**
 - Prelude's `snd` **which gives the second component of a pair may be defined as:-**
$$\text{snd} :: (a, b) \rightarrow b$$
$$\text{snd} \ (_, x) = x$$

Defining functions **CS1243**

- **Pattern matching**
- **List patterns**
 - **A list of patterns is itself a pattern, which matches any list of the same length whose elements match the corresponding patterns in order**
- **Examples:-**
 - **A function that determines a sequence of precisely 3 characters starting with a 'b' may be defined as:-**

```
stW :: [Char] -> Bool
stW ['b', _, _] = True
stW _ = False
```

NB use of the wildcard pattern



Defining functions

- **Pattern matching**
- **List patterns**
 - **A list is** `cons`**tructed one element at a time starting with the empty list** `[]` **using the operator** `:` **called** `cons`
 - **Example**
 - `[1, 2, 3]`
= `1:[2, 3]`
= `1:(2:[3])`
= `1:(2:(3:[]))`
 - **thus** `[1, 2, 3]` **is abbreviation of**
`1:(2:(3:[]))`
- `cons` **operator associates to the right**

Defining functions

- **Pattern matching**
- **List patterns**
 - **The stW could thus be defined as:-**

```
stW :: [Char] -> Bool
stW ['b':_] = True
stW _       = False
```

- **Could thus define null, head, tail as:-**

```
null :: [a] -> Bool
null [] = True
null (_:_) = False
```

```
head :: [a] -> a
head (x:_) = x
```

```
tail :: [a] -> [a]
tail (_:xs) = xs
```

CSI243

Defining functions

- **Pattern matching**
- **List patterns**
 - **NB** `cons` **patterns must be parenthesised as function application has higher priority than all other operators**

`tail _:xs = xs` **without parentheses means**
`(tail _):xs = xs`

which is both incorrect meaning and invalid definition

Defining functions **CS1243**

- **Pattern matching**
- **Integer patterns**
 - **Haskell supports integer patterns of the form $n+k$, where n is an integer variable and $k > 0$ is an integer constant**
 - **Could thus define the `pred` that maps 0 to itself and any positive integer to its predecessor as:-**

```
pred :: Int -> Int
pred 0 = 0
pred (n + 1) = n
```
 - **NB $n+k$ match integers $n > k$ in this definition**
 - **the proper use of parenthesis**

CSI243

Defining functions

- **Lambda (λ) expressions**
 - **Alternative to defining functions using equations**
 - **Functions can be defined using the λ expressions which comprises of:-**
 - **A pattern for each of the arguments**
 - **A body that specifies how the result can be computed in terms of the arguments**
 - **An anonymous function definition (as has no name)**

CSI243

Defining functions

- **Lambda (λ) expressions**
 - **nameless functions can still be used as the named ones**

- **Example**

- **Double a given number**

$\lambda x \bullet x + x$ **denoted in Haskell as** `\x -> x + x`

- **add two given numbers**

$\lambda x \ y \bullet x + y$ **denoted in Haskell as**

`\x y -> x + y`

- **Application**

$(\lambda x \bullet x + x) \ 3$ **denoted in Haskell as**

`(\x -> x + x) 3`

$(\lambda x \ y \bullet x + y) \ 8 \ 6$ **denoted in Haskell as**

`(\x y -> x + y) 8 6`

CSI243

Defining functions

- **Lambda (λ) expressions**
 - **Can be used to formalise the meaning of curried function definitions**
- **Example**
 - **add two given numbers**

`add x y = x + y` **can be understood to mean**

`add = \x -> (\y -> x + y)`

which makes precise that `add` is a function that takes a number `x` and returns a function, which in turn takes a number `y` and returns the results `x + y`

CSI243

Defining functions

- **Lambda (λ) expressions**
 - **also useful when defining functions that return functions as results by their nature, rather than as a consequence of curry**
- **Example**
 - **The `Prelude's` `const` that returns a constant function that always produces a given value may be defined as**

```
const :: a -> b -> a
const x = \_ -> x
```

Defining functions

- **Lambda (λ) expressions**
 - **useful in avoiding having to name a function that is only referenced once**
- **Example**
 - **The function `odds` that returns the first n odd integers may be defined as:-**

```
odds :: Int -> [Int]
odds n = map f [0..n-1]
  where
    f x = x * 2 + 1
```


Defining functions

- **Lambda (λ) expressions**
- **Example**
 - **The function `odds` may be simplified using λ -expressions to:-**

```
odds :: Int -> [Int]
```

```
odds n = map ( \x -> (x * 2) + 1) [0..n-1]
```

Defining functions **CS1243**

- **Sections**

- **Functions such as `+` are (infix) operators**
- **Any function with two arguments can be converted into an operator by enclosing its name in single back quotes. As in `5 `div` 2`. The converse is also possible**
- **Any operator can be converted into a curried function by enclosing its name in parenthesis as in `(+) 3 2`**
- **This may also be written as**
- **`(+3) 2` and (or)**
- **`(3+) 2`**

Defining functions **CS1243**

- **Sections**

- **In general, if $\oplus$ is an operator, then expressions of the form $(\oplus)$, $(x\oplus)$ and $(\oplus y)$ for arguments x and y are called sections**

- **The meaning of these formalised using λ -expressions:-**

- $(\oplus) = \lambda x \rightarrow (\lambda y \rightarrow x \oplus y)$

- $(x\oplus) = \lambda y \rightarrow x \oplus y$

- $(\oplus y) = \lambda x \rightarrow x \oplus y$

- **Sections can be used to construct a number of simple but useful functions in a compact way**

Defining functions **CS1243**

- **Sections**

- **Examples**

- **(+)** is the addition function

$$\lambda x \rightarrow (\lambda y \rightarrow x + y)$$

- **(1+)** is the successor function

$$\lambda y \rightarrow 1 + y$$

- **(1/)** is the reciprocal function

$$\lambda y \rightarrow 1 / y$$

- **(*2)** is the doubling function

$$\lambda x \rightarrow x * 2$$

- **(/2)** is the halving function

$$\lambda x \rightarrow x / 2$$

Defining functions

- **Sections**
 - **Sections necessary for stating the type of operators, as an operator itself is not a valid Haskell expression**
- **Example**
 - **$(\wedge)$ the logical conjunction**
$$(\wedge) :: \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$$
 - **$(\vee)$ the logical disjunction**
$$(\vee) :: \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}$$

Defining functions

- **Sections**
 - **Sections may also be necessary when using operators as arguments to other functions**
- **Example**
- **The function `and` that returns `True` if all logical values in a list are `True` may be defined as:-**
`and :: [Bool] -> Bool`
`and = foldr (∧) True`